WEEK 5

Sequential Logic

Flip-Flops & Registers

Circuit Elements · Flip-Flop Types · D Flip-Flop · Registers · Initialization · Reset · Blocking vs Non-blocking · Combinational + FF · Shift Register

AGENDA

What We'll Cover Today

1. Elements of a Sequential Circuit
2. Types of Flip-Flops (SR, D, JK, T)
3. The D Flip-Flop in Verilog
4. Building Registers from Flip-Flops
5. Initializing FFs & Registers
6. Reset Styles & Priority
7. Blocking vs Non-blocking
8. Combinational Logic + Flip-Flops
9. Shift Registers & DE0 Lab

1 · CONCEPT

From Combinational to Sequential Logic

Everything up to Week 4 had no memory. That changes today.

- Combinational logic: output depends only on the current inputs — no history, no memory.
- Sequential logic: output depends on current inputs AND the stored state from before.
- That stored state lives in flip-flops — the smallest unit of digital memory.
- Every register, counter, and FSM you'll build for the rest of the course is just flip-flops plus combinational logic around them.

Two new questions

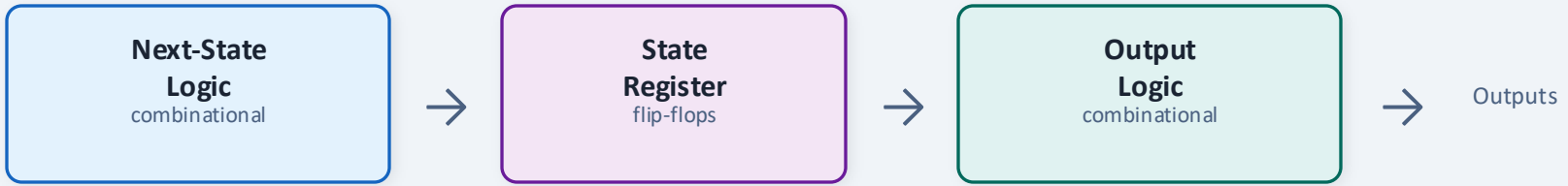
- When does state update? → on the clock edge
- What happens at start-up? → the reset

1 · CONCEPT

Elements of a Sequential Circuit

Three parts, working together in a loop

- Next-state logic — combinational logic that computes what the new state should be from the current state and inputs.
- State register — a bank of flip-flops (one per state bit) that stores the current state. This is the ONLY place memory exists.
- Output logic — combinational logic that computes the outputs from the state (and sometimes the inputs directly).
- The feedback path — the state register's output loops back into the next-state logic. That loop is what makes a circuit sequential instead of a straight combinational pipe.



The SR (Set-Reset) Flip-Flop

The oldest and simplest — and the reason D flip-flops exist

- S = 1 sets the output to Q = 1.
- R = 1 resets the output to Q = 0.
- S = 0, R = 0 holds the current value — no change.
- S = 1, R = 1 at the same time is invalid — the circuit is asked to set and reset simultaneously, and real hardware can't resolve which one wins.

S	R	Q (next)
0	0	hold (no change)
0	1	0
1	0	1
1	1	invalid / undefined

That invalid state is exactly why SR is rarely used directly in modern digital design — it can't guarantee predictable behavior.

JK and T Flip-Flops

JK fixes SR's invalid state; T is a simplified JK built for counting

- JK flip-flop: behaves like SR (J=Set, K=Reset), but when J=K=1 the output toggles instead of being undefined — no invalid state.
- T (Toggle) flip-flop: one input. T=1 flips the output every clock edge; T=0 holds. It's the natural building block for binary counters.
- Both still need two control inputs (JK) or a dedicated toggle input (T) — more circuitry than a plain D flip-flop needs.

J	K	Q (next)
0	0	hold (no change)
0	1	0
1	0	1
1	1	toggle (~Q)

JK truth table (left) — a T flip-flop is the special case where J and K are tied together as one input T: T=0 holds, T=1 toggles.

The D Flip-Flop

The fundamental 1-bit memory element — this is the one we actually build with

- Samples the D input the instant the clock rises (posedge) and holds that value on Q until the next rising edge.
- Between clock edges, D is completely ignored — Q simply holds its previous value.
- One D flip-flop stores exactly one bit. An 8-bit register is simply eight D flip-flops sharing one clock.



D — data in Q — data out CLK — clock (triggers on rising edge)

The D Flip-Flop in Verilog

always @(posedge clk or negedge rst_n) — clock and reset in one sensitivity list

VERILOG — dff.v

```
module dff (
  input  clk,
  input  rst_n,
  input  d,
  output reg q
);

  always @(posedge clk or negedge rst_n)
    begin
    if (!rst_n)
      q <= 1'b0;  // async reset
    else
      q <= d;         // sample D on posedge
  end
endmodule
```

Reading it

- posedge clk: react only on the rising edge
- negedge rst_n: react instantly when reset goes low
- rst_n is active-low — matches DE0's KEY buttons
- q is 'reg' because it's assigned inside always

Comparing All Flip-Flop Types

SR, D, JK, and T side by side — one reference table

Type	Inputs	Next-State Eq.	Toggle?	Invalid State?	Best For
SR	S, R	$Q^+ = S + R' \cdot Q$	No	Yes (S=R=1)	Legacy latch circuits
D	D	$Q^+ = D$	No	No	Universal — Verilog / FPGA design
JK	J, K	$Q^+ = JQ' + K'Q$	Yes	No	Textbook counters
T	T	$Q^+ = T \oplus Q$	Yes	No	Building block for counters

- Q^+ means 'the next value of Q, after the clock edge' — the standard notation for flip-flop behavior.
- Only D has no invalid state and no toggle ambiguity to design around — one clean reason it became the default (next slide).

3 · CHOOSING D

The Flip-Flop Chosen for HDL Design

Why every Verilog / VHDL design — including this course — builds exclusively with D flip-flops

- FPGA hardware only implements D flip-flops. The DE0's Cyclone III logic elements contain a D flip-flop and nothing else — there's no SR/JK/T primitive to instantiate even if you wanted one.
- always @(posedge clk) maps directly and unambiguously onto a D flip-flop — that's the exact hardware behavior the Verilog language was designed to model.
- No invalid state (unlike SR) and only one data input to reason about (simpler than JK's two) — fewer ways to write a bug.
- Need JK/T-style toggle behavior anyway? Just add a little combinational logic in front of a D flip-flop — same hardware, different next-state logic.

A T flip-flop, built entirely from a D flip-flop

```
always @(posedge clk or negedge rst_n)
  if (!rst_n)   q <= 1'b0;
  else if (t)      q <= ~q;  // D input = ~q  → toggle
  else
            q <= q;    // D input = q   → hold
```

Building an N-bit Register from Flip-Flops

A register is just N D flip-flops sharing one clock — one flip-flop per bit

- An 8-bit register = 8 independent D flip-flops, all clocked together, all reset together — Verilog just writes this as one 'reg [7:0]'.
- Two common patterns: a shift register (each bit feeds the next bit sideways — covered later this week) and a parallel-load register (all bits load at once from a wide input — used for ALU results, instruction registers, etc.).

VERILOG — reg8.v (parallel-load register)

```
module reg8 (
  input        clk, rst_n, en,
  input [7:0] d,
  output reg [7:0] q
);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 8'b0;
    else if (en)   q <= d;  // load all 8 bits at once
endmodule
```

Initializing a Single Flip-Flop

Three ways to set a starting value — only one works reliably on real hardware

1 · SIMULATION ONLY — never reaches the DE0

```
initial q = 1'b0;    // sets value only when ModelSim starts — Quartus ignores this for synthesis
```

2 · FPGA POWER-UP VALUE — synthesizable, but only happens once

```
reg q = 1'b0;   // Quartus bakes this into the bitstream's power-up state
```

3 · EXPLICIT RESET — repeatable, the real mechanism

```
always @(posedge clk or negedge rst_n)
  if (!rst_n) q <= 1'b0;   // works every time KEY[0] is pressed
```

Initializing Multi-bit Registers

Same three methods, just widened from 1 bit to N bits

- A register is N flip-flops, so the same three methods apply — just replace 1'b0 with an N-bit value like 8'h00.

8-bit register — all three methods, side by side

```
initial q = 8'h00;                  // 1: sim only — never reaches the DE0
reg [7:0] q = 8'h00;          // 2: FPGA power-up value — set once
always @(posedge clk or negedge rst_n)
  if (!rst_n) q <= 8'h00;          // 3: explicit reset — repeatable
```

Testbench tip: initialize registers to a distinctive pattern like 8'hAA (10101010) instead of all-zero — in the waveform it's instantly obvious whether a register was actually written or is still sitting at its uninitialized value.

Looking ahead — Week 11 (Memory Design) initializes whole arrays of registers (ROM/RAM) at once, with a for-loop in an initial block or $readmemh() loading values from a file. Same sim-only-vs-synthesizable distinction, just scaled up to many words.

PARAMETERS

What Is a Parameter?

A named constant, set once, that lets one module file serve many different configurations

- A parameter is a named constant fixed when the module is elaborated — it is not a signal and never changes while the circuit is running.
- Declared in the module header with #(parameter NAME = default_value), or with a plain parameter NAME = value; statement inside the module body.
- The point: write ONE module and reuse it at different sizes or settings, instead of copy-pasting a near-identical module for every width you need.
- Override a parameter when you instantiate the module — #(.NAME(new_value)) — leave it off and the default value is used instead.

VERILOG — one file, two widths

```
module counter #(parameter WIDTH = 8) (
  input clk, rst_n,
  output reg [WIDTH-1:0] count
);
...  endmodule

counter u1 (.clk(clk), .rst_n(rst_n), .count(cnt8));           // WIDTH=8  (default)
counter #(.WIDTH(32)) u2 (.clk(clk), .rst_n(rst_n), .count(cnt32));  // overridden to 32
```

PARAMETERS

Parameters for Register Width & Initial Value

Not just width — a parameter can also set what a register resets to

- Add a second parameter for the reset value, and one register file can describe any width AND any reset value — no more hand-editing the reset line inside near-duplicate files.
- This is exactly the register.v you'll reuse in the multi-stage pipeline example — here extended with an INIT parameter alongside its width parameter W.

VERILOG — register.v with W (width) and INIT (reset value)

```
module register #(
  parameter
 W    = 8,

parameter INIT = 0
) (
  input              clk, rst_n,
  input  [W-1:0]   d,
  output reg [W-1:0] q
);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) q <= INIT;  else q <= d;
endmodule
```

Synchronous vs Asynchronous Reset

Rule of thumb for this course: always write explicit reset logic — never rely on power-up values alone

	Synchronous	Asynchronous
Sensitivity list	@(posedge clk) only	@(posedge clk or negedge rst_n)
Takes effect	Only on the next clock edge	Immediately, regardless of clock
Good for	Clean timing closure, filters glitches	Guaranteed known state at power-up
DE0 KEY button	Works, but only clears on next clk	Instant clear — matches button-press feel

6 · RESET

The Standard Reset-Priority Pattern

Reset always wins — check it first, every time

VERILOG — priority pattern

```
always @(posedge clk or negedge rst_n) begin
  if (!rst_n)
    q <= 0;          // 1: reset wins over all
  else if (en)
    q <= next_value; // 2: updates only if enabled
  else
    q <= q;          // 3: hold (often implicit)
end
```

The #1 mistake

- Checking 'en' before 'rst_n' means a held enable can block reset from ever taking effect.
- Always test reset first, then enable, then default hold — in that order.

7 · ASSIGNMENT

What Is Blocking vs Non-blocking Assignment?

Two assignment operators inside always blocks — same-looking symbols, very different meaning

- Blocking (=): executes immediately, in the order written — like an ordinary line of code. The next statement waits ('blocks') until this one finishes.
- Non-blocking (<=): schedules the assignment for the end of the current time step. Every right-hand side in the block is read first using the OLD values, then all left-hand sides update together, at once.
- Mental model: blocking = 'do this now, then move to the next line.' Non-blocking = 'calculate now using what everything was, apply everywhere simultaneously, later.'

Same two lines, opposite result

```
// Blocking ( = ) — sequential, like a C statement
a = 1;
b = a;     // b becomes 1 — a was already updated above

// Non-blocking ( <= ) — parallel, like real hardware
a <= 1;
b <= a;    // b becomes the OLD a — not 1 yet, updates together
```

7 · ASSIGNMENT

Blocking (=) vs Non-blocking (<=)

The single most common source of sequential-logic bugs

	Blocking (=)	Non-blocking (<=)
Executes	Immediately, in program order	Scheduled — all RHS read first, updates applied together
Used in	Combinational always (@*), assign	Sequential always (@posedge clk)
Models	A wire — instant update	A flip-flop — updates on the clock edge
Mixing them	Don't mix = and <= in one always block	Keeps every register update simultaneous & predictable

7 · ASSIGNMENT

Blocking vs Non-blocking — Real Design Scenarios

Rule of thumb: combinational always (@*) → blocking. Sequential always (@posedge clk) → non-blocking.

Combinational ALU — blocking is correct

```
always @(*) begin
  case (op)
    2'b00: result = a + b;
    2'b01: result = a - b;
    default: result = 0;
  endcase
end
// result is a wire-like value —
// blocking makes it update instantly
```

FSM State Register — non-blocking is correct

```
always @(posedge clk or negedge rst_n)
  if (!rst_n) state <= IDLE;
  else
        state <= next_state;

// next_state: combinational logic
// (Week 7-8 FSM) reading the OLD
// 'state' + inputs. Non-blocking keeps
// that read consistent for every line
// in this clock edge — no risk of
// 'state' changing mid-calculation
```

7 · ASSIGNMENT

Why the Operator Choice Matters

A 3-stage shift register — same code, one character different

WRONG — blocking (=)

```
always @(posedge clk)
  begin
  q1 = d;
  q2 = q1;    // q1 already = d!
  q3 = q2;    // all 3 = d, same cycle
end
```

CORRECT — non-blocking (<=)

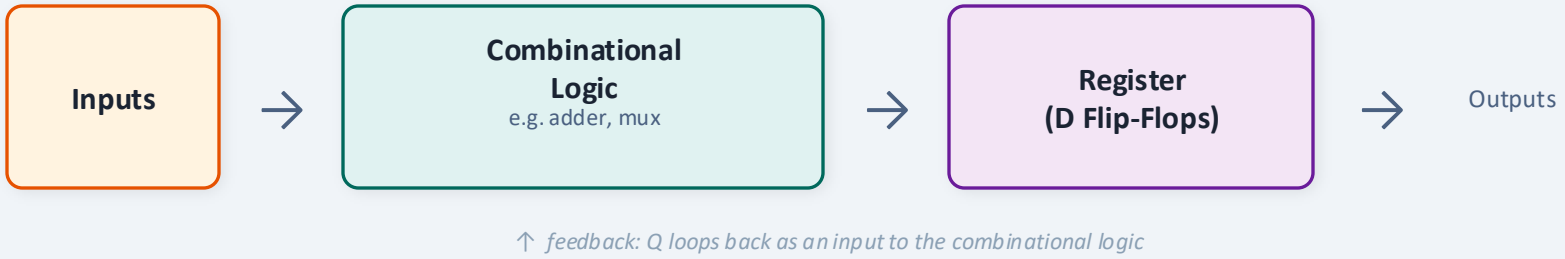
```
always @(posedge clk)
  begin
  q1 <= d;
  q2 <= q1;  // old q1, one stage
behind
  q3 <= q2;  // old q2 — proper
shifting!
end
```

8 · COMB + FF

Combining Flip-Flops with Combinational Logic

Every real register is combinational logic feeding a flip-flop, looped back on itself

- The general pattern: next_state <= combinational_function(current_state, inputs); — written inside a single clocked always block.
- The flip-flop's own output (Q) is almost always one of the inputs to the combinational logic in front of it — that feedback is what lets state build up over time instead of just holding one value.
- This exact pattern is what turns a plain register into a counter, an accumulator, or an FSM — only the combinational function in front changes.



Example: Accumulator

A running sum — the simplest 'combinational logic + flip-flop' circuit you can build

VERILOG — accumulator.v

```
module accumulator (
  input        clk, rst_n, en,
  input [7:0] a,
  output reg [7:0] acc
);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) acc <= 8'b0;
    else if (en)   acc <= acc + a;  // comb '+' feeds the register
endmodule
```

acc + a is pure combinational logic (an adder); acc <= … is the register capturing the result each clock — that's why acc keeps growing instead of just holding a.

Shift Registers

A chain of D flip-flops, each one feeding the next

- Every clock edge, each flip-flop copies the value of the one before it — the whole pattern shifts one position.
- The first flip-flop takes a new bit from outside (serial_in); the last flip-flop's value is lost off the end.
- An enable signal lets you freeze the register when you don't want it to shift on that cycle.
- Used for: serial-to-parallel conversion, delay lines, and LED 'chasing light' patterns — exactly what we'll test on the DE0 today.

8-bit Shift Register with Enable

This week's lab module — combine D flip-flop behavior with enable and concatenation

VERILOG — shift_reg8.v

```
module shift_reg8 (
  input        clk, rst_n, en, serial_in,
  output [7:0] q
);
  reg [7:0] q_reg;

  always @(posedge clk or negedge rst_n)
    if (!rst_n)
      q_reg <= 8'b0;
    else if (en)
      q_reg <= {q_reg[6:0], serial_in};  // shift left, insert new bit

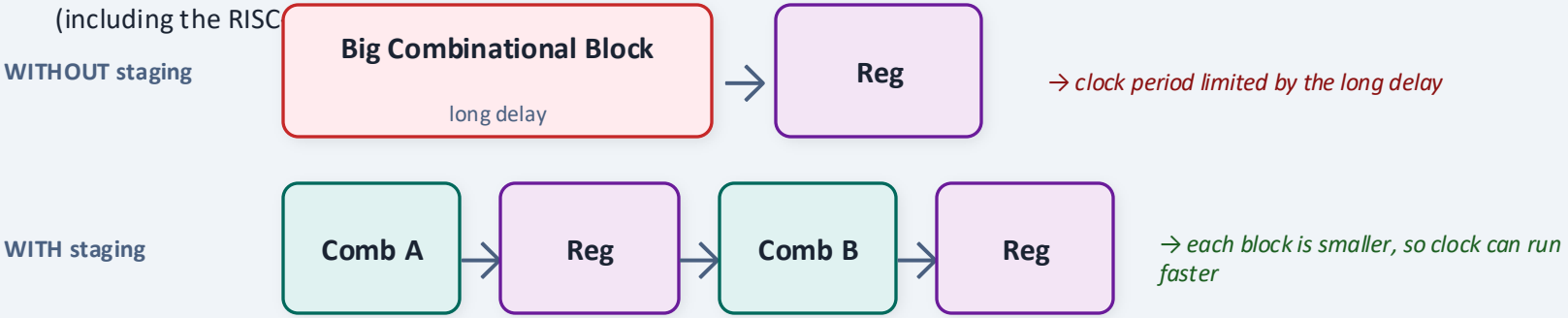
  assign q = q_reg;
endmodule
```

MULTI-STAGE EXAMPLE

Why Split Combinational Logic Into Stages?

The problem that one register in the middle solves

- A single big combinational block has a total delay equal to every gate delay along its longest path — the clock period must be at least that long.
- Split the same logic into two smaller blocks with a register in between, and each block only has to finish within one (shorter) clock period — the clock can run much faster.
- The catch: an input now takes 2 clock cycles to reach the final output instead of 1. That delay is called latency; the payoff is far higher throughput once the pipeline is full.
- This technique — splitting combinational logic into stages separated by registers — is called pipelining, and it's exactly how real CPUs (including the RISC



MULTI-STAGE EXAMPLE

The Building Block: One Stage

Combinational logic + the register that captures its result

- A stage takes its input, computes something purely combinational, and a register grabs the answer on the next clock edge.
- That register's output IS the input to the next stage — chain enough of these together and you get a pipeline.
- This is nothing new: it's the exact same 'combinational logic feeding a register' pattern from the Combining Flip-Flops slide earlier this week — a multi-stage circuit is just that pattern, repeated.



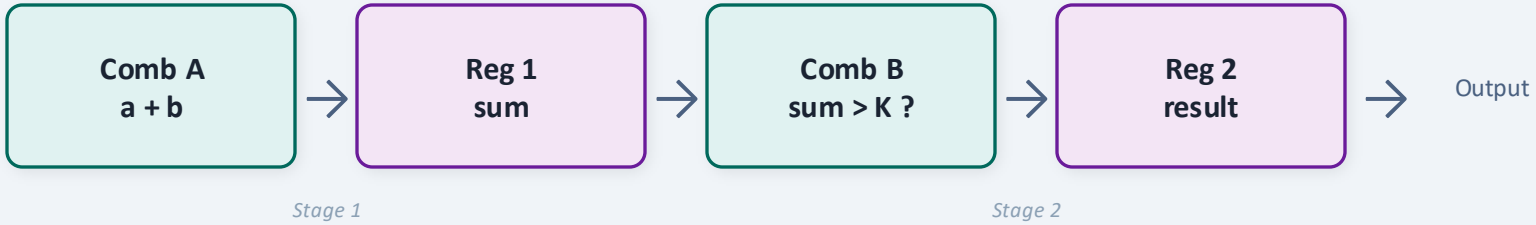
"One stage"

MULTI-STAGE EXAMPLE

Our Example: Two Stages Combined

Stage 1 computes a sum; Stage 2 makes a decision from it

- Stage 1: Comb A adds two numbers (a + b); Reg 1 captures the sum.
- Stage 2: Comb B compares that sum against a threshold K; Reg 2 captures the 1-bit result.
- Reg 1 sits physically between the two combinational blocks — it's simultaneously 'the end of Stage 1' and 'the start of Stage 2.'



MULTI-STAGE EXAMPLE

The Verilog Code, Split Into Files

Each block in the diagram becomes its own file — exactly like dff.v and shift_reg8.v earlier this week

- Real projects rarely put everything in one module. Each combinational block and each register gets its own file, with one clear job.
- Reg 1 and Reg 2 are both just N-bit registers — so one reusable register.v (parameterized by width) can serve as both, instead of writing the same always block twice.
- A top-level file then wires the pieces together in the same order as the diagram: adder → register → comparator → register.

File	Implements	Type
adder.v	Comb A — sum = a + b	Combinational
register.v	Reg 1 and Reg 2 (reused, width set by parameter W)	Sequential
comparator.v	Comb B — result = (sum > K)	Combinational
two_stage_pipe.v	Top-level — instantiates and wires the three files above	Structural

MULTI-STAGE EXAMPLE

The Leaf Modules

adder.v and comparator.v do the math; register.v is reused for both pipeline registers

adder.v

```
module adder (
  input [7:0] a, b,
  output [8:0] sum
);
  assign sum = a + b;
endmodule
```

comparator.v

```
module comparator (
  input [8:0] sum,
  input [7:0] K,
  output result
);
  assign result = (sum > K);
endmodule
```

register.v — reusable N-bit register, used for both Reg 1 (W=9) and Reg 2 (W=1)

```
module register #(parameter W = 8) (
  input              clk, rst_n,
  input  [W-1:0]   d,
  output reg [W-1:0] q
);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 0;  else q <= d;
endmodule
```

MULTI-STAGE EXAMPLE

two_stage_pipe.v — Wiring the Stages Together

The top-level file instantiates the three leaf modules in the same order as the diagram

VERILOG — two_stage_pipe.v (structural / top-level)

```
module two_stage_pipe (
  input        clk, rst_n,
  input [7:0] a, b, K,
  output       result
);
  wire [8:0] sum_comb, sum_reg;
  wire       result_comb;
  adder u_adder (.a(a), .b(b), .sum(sum_comb));
  register #(.W(9)) u_reg1 (
    .clk(clk), .rst_n(rst_n),
    .d(sum_comb), .q(sum_reg));

  comparator u_cmp (
    .sum(sum_reg), .K(K), .result(result_comb));

  register #(.W(1)) u_reg2 (
    .clk(clk), .rst_n(rst_n),
    .d(result_comb), .q(result));
endmodule
```

Reading it

- No always block here at all — this file only connects modules, it doesn't compute anything itself.
- register is instantiated twice with a different W each time — that's the reuse paying off.
- sum_comb → u_reg1's input, sum_reg → u_cmp's input: the wire names show exactly where the pipeline boundary sits.

MULTI-STAGE EXAMPLE

Walking Through the Clock Cycles

Tracing a=3, b=7, K=8 — watch the 1-cycle gap between sum and result

Clock Edge	a, b	sum (Reg 1, after this edge)	result (Reg 2, after this edge)
Reset	—	0	0
Edge 1	3, 7	10 (3+7, brand new)	0 (uses OLD sum = 0)
Edge 2	3, 7	10 (unchanged)	1 (uses sum = 10 from Edge 1)
Edge 3	3, 7	10 (steady state)	1 (steady state)

result only becomes 1 at Edge 2 — a full clock cycle after sum first became 10 at Edge 1. That one-cycle gap is the pipeline's latency: the price paid for letting each stage's combinational logic stay short.

MULTI-STAGE EXAMPLE

Why This Matters

This 2-stage toy example is exactly how the Week 10 RISC-V pipeline works

- The RISC-V core you'll build in Week 10 uses this same idea, just with more stages: Fetch → Decode → Execute → Memory → Writeback — five combinational blocks, each with its own pipeline register in between.
- Every one of those pipeline registers is a plain D flip-flop bank, updated with non-blocking assignment — exactly like sum and result_r here.
- Get comfortable with this 2-stage example now, and the 5-stage RISC-V pipeline in Week 10 will look like 'the same trick, just longer.'



Week 10 preview — RISC-V single-cycle pipeline, 5 stages

DE0 Pin Mapping for This Week

Everything the top-level wrapper needs to connect

DE0 Pin	Connects to	Purpose
CLOCK_50	clk	50 MHz onboard oscillator
KEY[0]	rst_n (active-low)	Asynchronous reset
KEY[1]	en	Enable — hold to freeze shifting
SW[0]	serial_in	Bit fed into the register each shift
LEDG[7:0]	q[7:0]	Watch the bit move across the LEDs

Lab Walkthrough

Build, wire, program, test

1. Create a new Quartus project and add dff.v and shift_reg8.v from today's slides.
2. Write a top-level wrapper module that instantiates shift_reg8 and maps its ports to DE0 pins.
3. Assign pins per the table on the previous slide, then Compile in Quartus.
4. Program the DE0 via USB-Blaster.
5. Hold KEY[1] (enable) and toggle SW[0] between presses — watch the bit shift across LEDG[7:0].

COMMON MISTAKES

Watch Out For These

The bugs that show up every year in this lab

- Mixing = and <= in the same always block — pick one style per block, always <= for sequential.
- Reversing the if/else priority — check reset first, then enable, never the other way around.
- Forgetting rst_n is active-low — if (!rst_n), not if (rst_n).
- Relying on an initial block or a declaration-time value ('reg q = 0') for reset behavior — neither one fires again when KEY[0] is pressed; write explicit reset logic instead.
- Using = instead of <= in a shift chain — every stage silently becomes the same value in one cycle.

GEMINI

Gemini Prompts for This Week

Paste your .v file and ask directly

1. "My flip-flop's output never updates — here is my always block, what's wrong?" (paste your dff.v)
2. "Explain the difference between = and <= in Verilog with a simple 3-line example."
3. "What's the difference between synchronous and asynchronous reset, and which should I use for my DE0 lab?"
4. "Generate an 8-bit shift register with an enable input and asynchronous active-low reset, in Verilog (not VHDL)."
5. "Show me how to combine an adder with a D flip-flop to build a simple accumulator in Verilog."

Week 5 Summary

A sequential circuit = next-state logic + state register + output logic, wired in a feedback loop ✓

D flip-flops are what real hardware builds with — SR/JK/T are conceptual, not separate FPGA hardware ✓

An N-bit register is N D flip-flops sharing one clock — shift or parallel-load, same flip-flops ✓

Only explicit reset logic reliably re-initializes state — initial blocks and power-up values don't repeat, for single bits or whole registers ✓

Non-blocking (<=) models real flip-flop behavior; reset must be checked before enable ✓

Combinational logic + a register, looped back on itself, is the pattern behind counters, accumulators, and FSMs ✓

Built and tested an 8-bit shift register live on the DE0 ✓

▶ Week 6: Counters & Shift Registers — a 4-bit up/down counter driving the DE0's 7-segment display